École Nationale Supérieure d'Arts et Métiers – Meknès
Filière Intelligence Artificielle – 4e année

Cahier des Charges

Sawti-ASR-FineTuner & Data-Augmenter

Pipeline ASR & MLOps Audio pour la Darija marocaine
et le code-switching français–arabe

Réalisé par : Mohamed Amine Belasri
Yahya Amajane

Encadré par : Pr. Hosni
Pr. Menaoui

Année universitaire 2026–2027

Table des matières

1 Présentation du projet **4**
1.1 Titre du projet 4
1.2 Problème à résoudre 4
1.3 Objectif de la solution 4
1.4 Périmètre 5
1.5 Utilisateurs visés 5

2 Exigences fonctionnelles **6**
2.1 Fonctionnalités principales 6
2.2 Entrées attendues 6
2.3 Sorties attendues 6
2.4 Scénarios d'utilisation 7
2.5 Priorité des fonctionnalités 7

3 Diagrammes UML **8**
3.1 Diagramme de cas d'utilisation 8
3.2 Diagramme d'activité 9
3.3 Diagramme de séquence 10
3.4 Diagramme d'états-transitions 10
3.5 Diagramme de classes conceptuel 11

4 Données **12**

5 Approche IA **13**

6 Exigences non fonctionnelles **14**

7 Planning et livrables **15**

8 Annexes **16**

Liste des tableaux

1.1 Matrice d'inclusion et d'exclusion du projet 5

2.1 Fonctionnalités principales 6
2.2 Priorité des fonctionnalités 7

Table des figures

3.1 Diagramme de cas d'utilisation 8
3.2 Diagramme d'activité du pipeline 9
3.3 Diagramme de séquence de l'expérience complète 10
3.4 Diagramme d'états-transitions d'une version de modèle 10
3.5 Diagramme de classes conceptuel 11

Chapitre 1

Présentation du projet

1.1 Titre du projet

Sawti-ASR-FineTuner & Data-Augmenter – Pipeline ASR & MLOps Audio.

Il s'agit d'un projet d'initiation de 4e année (filière IA, ENSAM Meknès), réalisé en binôme sur un semestre (16 semaines), dans le cadre de la plateforme Sawti (thème : continuité, Arabizi et code-switching).

1.2 Problème à résoudre

Les modèles fondationnels de reconnaissance automatique de la parole (ASR), comme Whisper, sont entraînés principalement sur des langues bien dotées en données. Ils donnent de mauvais résultats sur la Darija marocaine, pour trois raisons :

- **Rareté des données** : il existe peu de corpus de Darija transcrits.
- **Code-switching** : les locuteurs alternent la Darija et le français dans une même phrase.
- **Conditions réelles** : les enregistrements contiennent du bruit de fond (rue, café, téléphone), absent des corpus propres.

De plus, adapter ces modèles par un *full fine-tuning* coûte cher en calcul, alors que le projet ne dispose que de GPU gratuits (Kaggle et Google Colab, T4/P100).

1.3 Objectif de la solution

Le projet vise à concevoir un pipeline MLOps automatisé qui :

1. augmente les données audio (SpecAugment, injection de bruit, variations de pitch et de vitesse) pour compenser leur rareté ;
2. fine-tune efficacement le modèle Whisper avec LoRA (PEFT) ;
3. évalue la robustesse du modèle sur trois jeux de test : propre, bruité et code-switching ;
4. trace toutes les expériences et versionne les modèles avec MLflow.

L'objectif mesurable est de battre la baseline (full fine-tuning sans augmentation ni PEFT) en gain relatif de WER. Les valeurs cibles sont données dans la partie 5.

1.4 Périmètre

Le tableau 1.1 précise ce que le projet fait et ne fait pas.

Inclus dans le projet	**Exclu du projet**
Préparation des données audio au format Hugging Face Datasets (nettoyage, rééchantillonnage à 16 kHz, découpage train/validation/test).	Collecte ou enregistrement de nouveaux audios sur le terrain.
Pipeline d'augmentation acoustique et spectrale avec niveaux de bruit contrôlés (SNR).	Transcription manuelle à grande échelle de nouvelles données.
Baseline : Whisper zero-shot et full fine-tuning (version réduite compatible avec un GPU T4).	Interface utilisateur ou application finale de transcription.
Fine-tuning LoRA/PEFT et étude de ses hyperparamètres (rang, modules ciblés).	Déploiement en production (serveur, API temps réel).
Évaluation WER/CER sur trois jeux de test et calcul du gain relatif.	Entraînement de grands modèles (Whisper medium/large) ou depuis zéro.
Suivi des expériences, registre de modèles (MLflow) et script de réentraînement automatisé.	Synthèse vocale (TTS) et traduction.

Table 1.1 – Matrice d'inclusion et d'exclusion du projet

1.5 Utilisateurs visés

— **Équipe de la plateforme Sawti et encadrants** : consultent les résultats et comparent les modèles.

— **Ingénieurs ML / data scientists** : lancent le pipeline, ajoutent des données et réentraînent les modèles.

— **Communauté open source** (utilisateurs secondaires) : réutilisent le code publié sur GitHub et le modèle publié sur Hugging Face.

Chapitre 2

Exigences fonctionnelles

2.1 Fonctionnalités principales

ID	Fonctionnalité	Description
F1	Préparation des données	Charger les corpus, nettoyer, rééchantillonner à 16 kHz, normaliser les transcriptions, découper en train/validation/test.
F2	Augmentation	Ajouter du bruit (ESC-50, NOISEX) à un SNR contrôlé, faire varier le pitch et la vitesse, appliquer SpecAugment.
F3	Baseline	Évaluer Whisper en zero-shot et réaliser un full fine-tuning sans augmentation.
F4	Fine-tuning LoRA	Entraîner des adaptateurs LoRA sur Whisper, avec ou sans augmentation.
F5	Évaluation	Calculer le WER et le CER sur les jeux propre, bruité et code-switching, puis le gain relatif par rapport à la baseline.
F6	Suivi des expériences	Enregistrer dans MLflow les paramètres, les métriques et les artefacts de chaque expérience.
F7	Registre de modèles	Versionner les modèles et identifier la meilleure version.
F8	Réentraînement automatisé	Relancer toute la chaîne avec une seule commande et un fichier de configuration.
F9	Publication	Publier le meilleur modèle et sa documentation sur Hugging Face.

Table 2.1 – Fonctionnalités principales

2.2 Entrées attendues

— Fichiers audio (WAV ou FLAC, mono, 16 kHz) accompagnés de leur transcription.
— Corpus de bruits (ESC-50, NOISEX) pour l'augmentation.
— Fichier de configuration YAML : modèle, hyperparamètres, niveaux de SNR.
— Checkpoints existants, pour reprendre un entraînement interrompu.

2.3 Sorties attendues

— Modèles fine-tunés et adaptateurs LoRA (checkpoints).

— Transcriptions prédites sur les jeux de test.
— Métriques WER et CER par jeu de test, et tableau comparatif des expériences.
— Runs MLflow (paramètres, courbes, artefacts) et versions de modèles dans le registre.

2.4 Scénarios d'utilisation

Scénario 1 – Expérience complète. L'ingénieur rédige un fichier de configuration et lance le pipeline. Les données sont préparées puis augmentées, le modèle est entraîné avec des checkpoints réguliers, puis évalué sur les trois jeux de test. Les résultats sont enregistrés dans MLflow. Si le modèle est meilleur que la version actuelle, il est ajouté au registre.

Scénario 2 – Réentraînement avec de nouvelles données. De nouvelles données sont ajoutées au corpus. L'ingénieur relance la même commande. Une nouvelle version du modèle est produite et comparée à l'ancienne.

Scénario 3 – Reprise après une coupure du GPU. La session Kaggle ou Colab s'arrête pendant l'entraînement. L'ingénieur relance le pipeline, qui reprend au dernier checkpoint sauvegardé.

Scénario 4 – Comparaison des expériences. L'encadrant ouvre MLflow et compare les runs (ablations : avec ou sans augmentation, LoRA ou full fine-tuning, différents rangs et niveaux de SNR).

2.5 Priorité des fonctionnalités

ID	Fonctionnalité	Priorité
F1	Préparation des données	Indispensable
F2	Augmentation	Indispensable
F3	Baseline	Indispensable
F4	Fine-tuning LoRA	Indispensable
F5	Évaluation	Indispensable
F6	Suivi des expériences	Importante
F7	Registre de modèles	Importante
F8	Réentraînement automatisé	Importante
F9	Publication	Optionnelle

TABLE 2.2 – Priorité des fonctionnalités

Chapitre 3

Diagrammes UML

3.1 Diagramme de cas d'utilisation

L'ingénieur ML utilise toutes les fonctions du pipeline. L'encadrant consulte et compare les résultats. MLflow est un acteur système qui stocke les expériences et les versions de modèles.

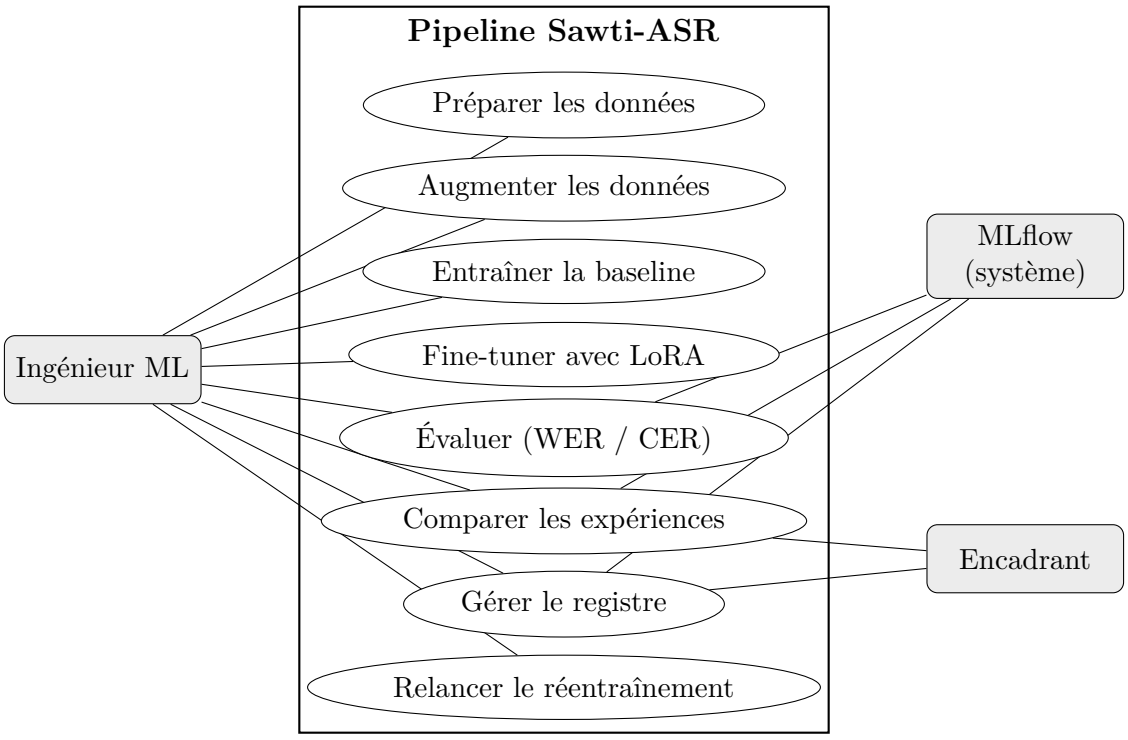


FIGURE 3.1 – Diagramme de cas d'utilisation

3.2 Diagramme d'activité

Le diagramme suivant décrit le déroulement d'une exécution complète du pipeline.

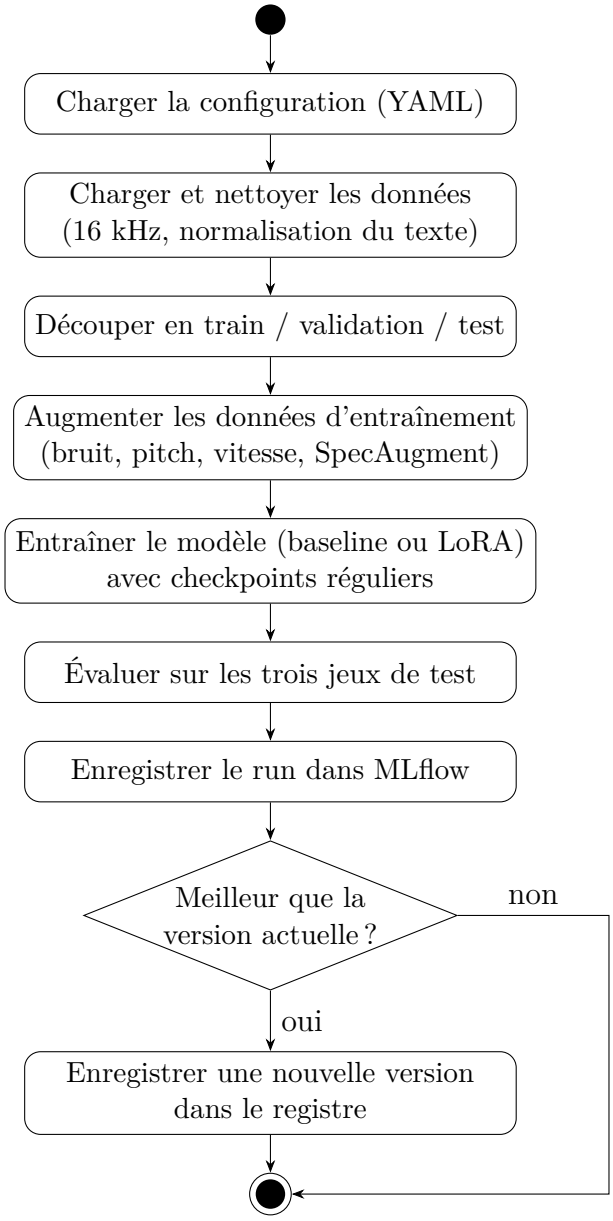


FIGURE 3.2 – Diagramme d'activité du pipeline

3.3 Diagramme de séquence

Le scénario principal est l'expérience complète (scénario 1).

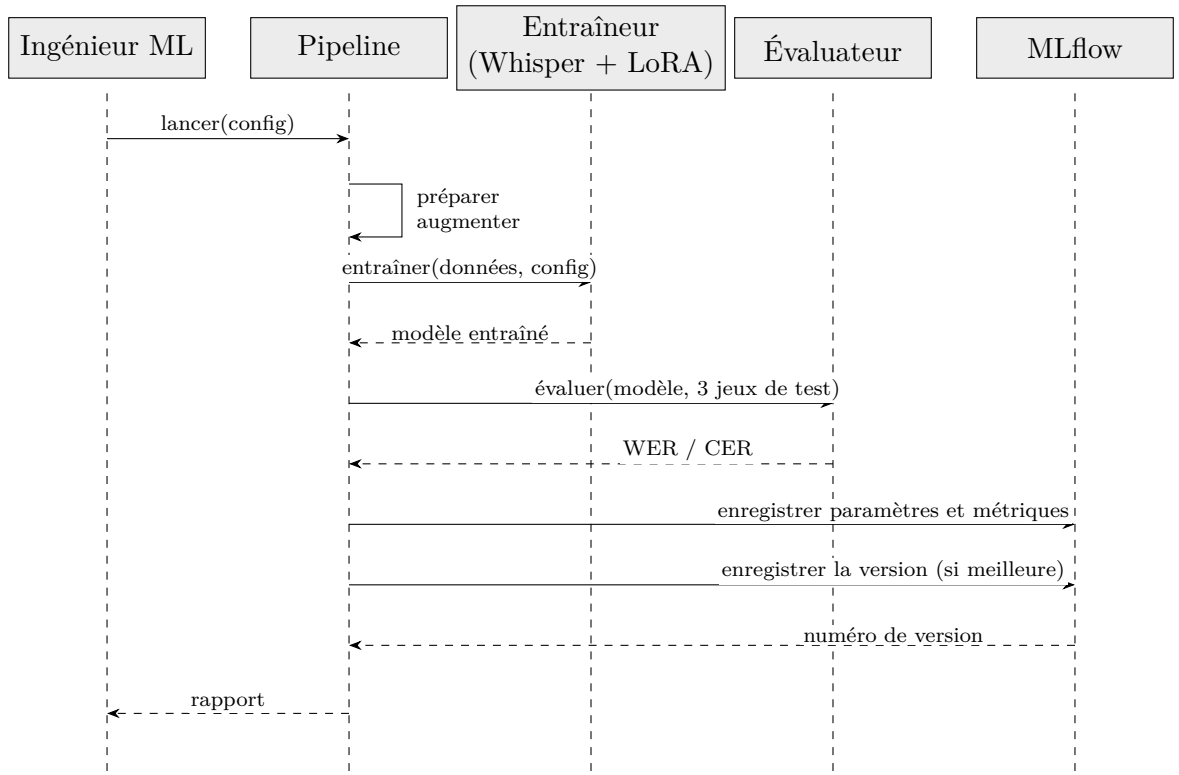


FIGURE 3.3 – Diagramme de séquence de l'expérience complète

3.4 Diagramme d'états-transitions

L'entité qui change d'état est la **version de modèle** : elle est entraînée, évaluée, puis rejetée ou promue, et enfin archivée lorsqu'une meilleure version la remplace. Dans MLflow, ces états seront représentés par des étiquettes ou des alias de version.

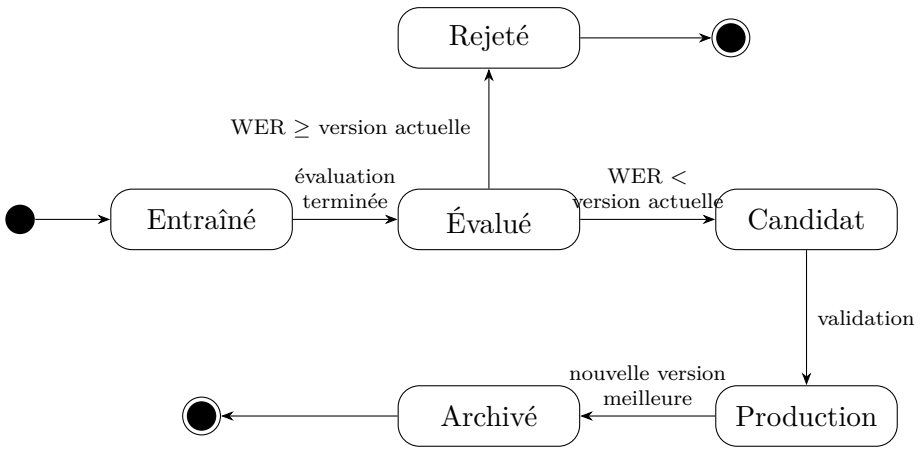


FIGURE 3.4 – Diagramme d'états-transitions d'une version de modèle

3.5 Diagramme de classes conceptuel

Le diagramme présente les entités métier du projet et leurs relations.

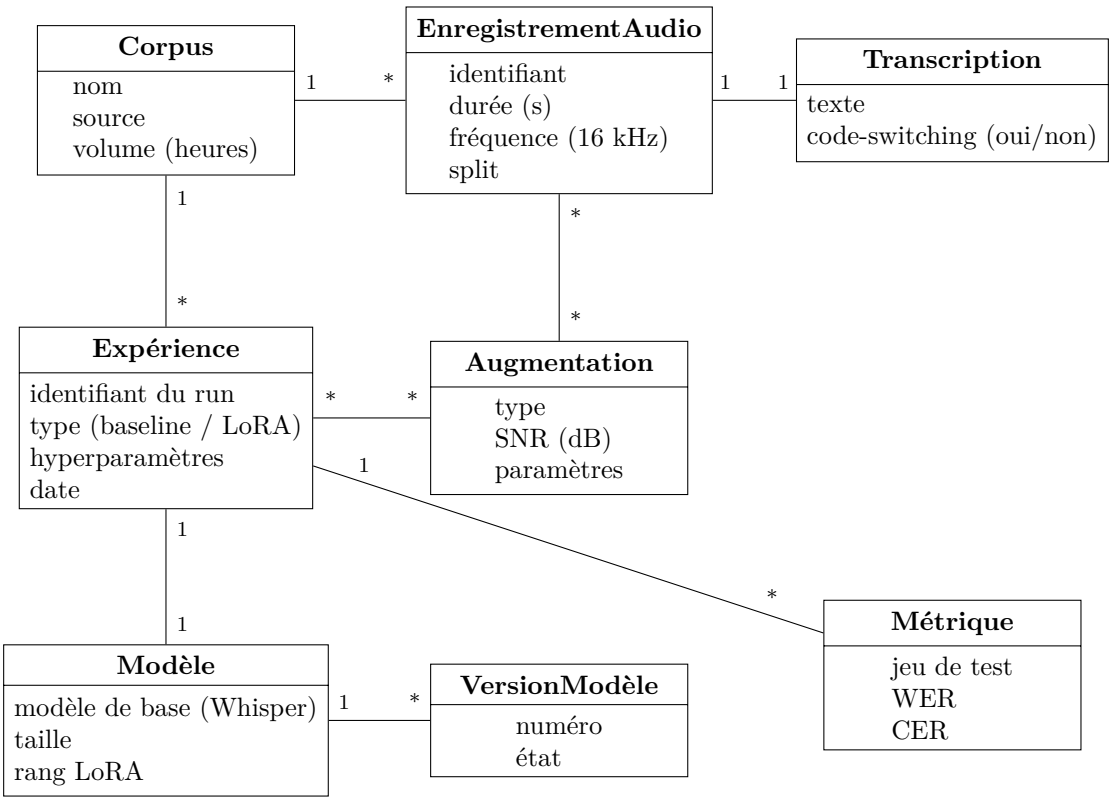


FIGURE 3.5 – Diagramme de classes conceptuel

Chapitre 4

Données

(À rédiger.)

Chapitre 5

Approche IA

(À rédiger.)

Chapitre 6

Exigences non fonctionnelles

(À rédiger.)

Chapitre 7

Planning et livrables

(À rédiger.)

Chapitre 8

Annexes

(À rédiger.)